Building a System

1.Understanding the System

Before anything else, you need to know what system you are building.

Example: E-Commerce System

Identify:

The main operations the system will perform (e.g., managing users, products, orders, payments)

The primary users of the system

The expected outcomes you want to achieve

2.Understanding Basic Database Concepts

You must understand the fundamentals of databases before implementing anything:

Tables / Entities → which table is primary and which is dependent

Fields / Columns → data types for each field (String, Number, Date, Boolean,Reference)

Relationships → 1:N, N:M, Foreign Keys / Reference Fields

Primary Keys → unique identifier for each record

Constraints → Not Null, Unique, etc.

3.Designing the ERD

Once you understand the system and the basic database concepts:

Draw the ERD for all tables, fields, and relationships

Identify Primary Keys and Foreign Keys / Reference Fields

The ERD will serve as your blueprint for building the system in ServiceNow

Requirements – E-Commerce System

A client has an online store and wants to create a system that allows customers to interact with products easily:

The client has customers, and each customer has a profile containing basic information such as name, email, and phone number.

The client has products for sale, and each product has a price, description, and image, as well as a category that helps customers browse and search.

Each product is stored in a specific warehouse, and it’s important to know the available quantity in each warehouse.

Requirements - E-Commerce System

When a customer decides to make a purchase, a bill/invoice is generated for each transaction. The invoice includes the customer, the products purchased, quantity, total price, and additional information such as payment method and date.

The system should be able to:

Link customers to the products they purchase via invoices

Link products to warehouses to track available stock

Link products to categories to facilitate search and classification

- In short, the system connects customers with products, tracks stock and categories, and records every purchase in an invoice.

1.Understanding the System

You want to register products:

Store information like product name, price, description, image, and category.

Track which warehouse each product is stored in and the available quantity.

You want to register customers:

Store customer information such as name, email, and phone number.

Keep track of their purchases through invoices.

1.Understanding the System

You want to track purchases and invoices:

Each invoice records which customer bought which products, in what quantity, the total price, and payment details.

You want to categorize products:

Each product belongs to a category to make searching and browsing easier for customers.

In short, the system should allow registering products, customers, and their purchases, while tracking stock and categorization.

2. Understanding Basic Database Concepts

Before creating the system in ServiceNow, you need to understand the basic database concepts that will help structure the data correctly:

Tables / Entities

Think of each main object in the system as a table:

Customers → store customer information

Products → store product details

Invoices → store purchase records

Warehouses → store stock locations

Categories → organize products

2.Understanding Basic Database Concepts

Fields / Columns

Each table contains fields to store data. For example:

Product table: Name, Price, Description, Image, Category reference

Customer table: Name, Email, Phone number

2. Understanding Basic Database Concepts

Relationships:

1:N (One-to-Many): A customer can have many invoices, an invoice can have many products (through invoice items).

N:1 (Many-to-One): Each product belongs to one category.

M:N (Many-to-Many): Products and invoices are linked through invoice items.

2.Understanding Basic Database Concepts

Primary Keys and Reference Fields:

Every table should have a unique identifier (Primary Key)

Reference Fields link one table to another (like foreign keys)

Constraints:

Some fields may be mandatory, unique, or have default values.

Example: Product Name should be unique, Quantity cannot be negative.

By understanding these concepts, you know how to structure your data properly before creating the ERD and implementing the system in ServiceNow.

3.Designing the ERD

Before we move on to the third step, let's make sure we understand what an ERD is and why it is important.

What is ERD? (Entity Relationship Diagram)

What is ERD? (Entity Relationship Diagram)

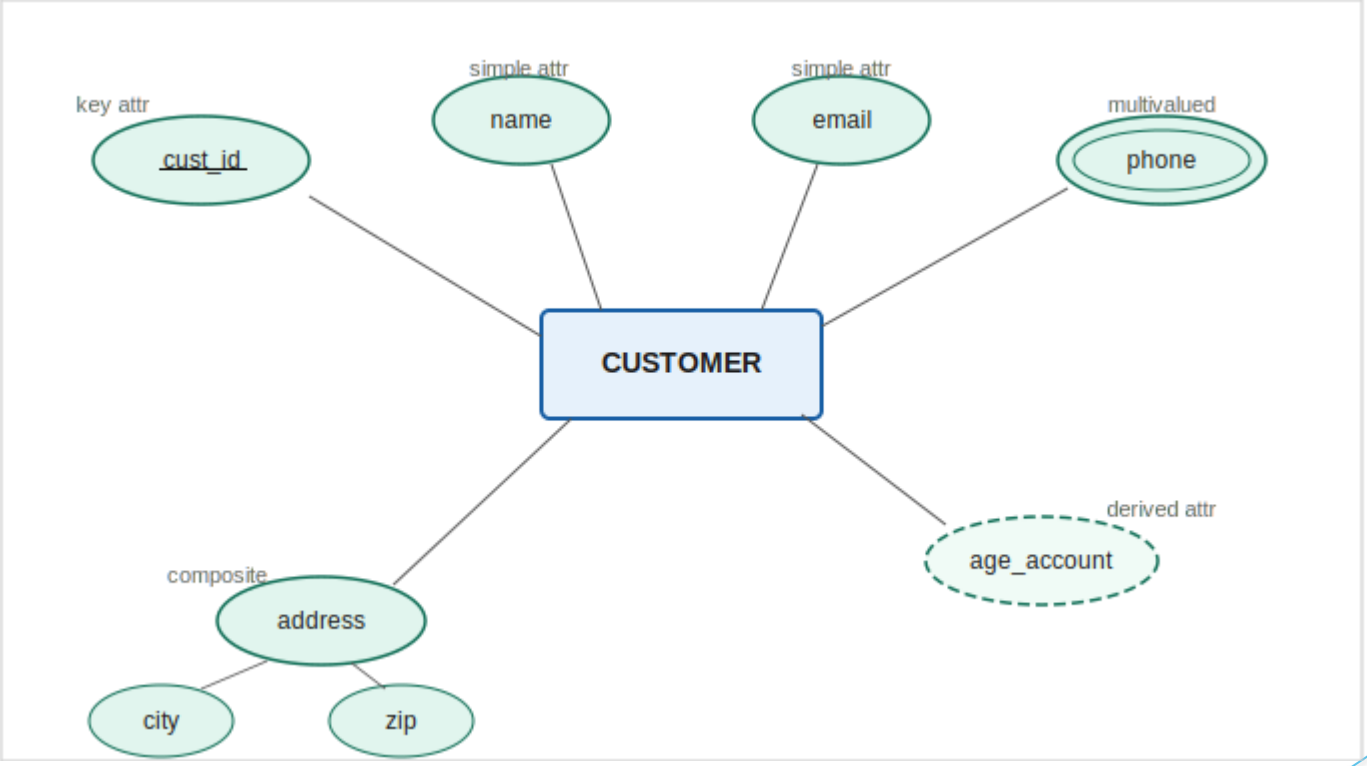
Definition:

An Entity Relationship Diagram (ERD) is a visual representation of the entities (tables) in a system and the relationships between them.

It helps to understand how data is structured before creating the actual database.

Types of ERD (Entity-Relationship Diagrams)

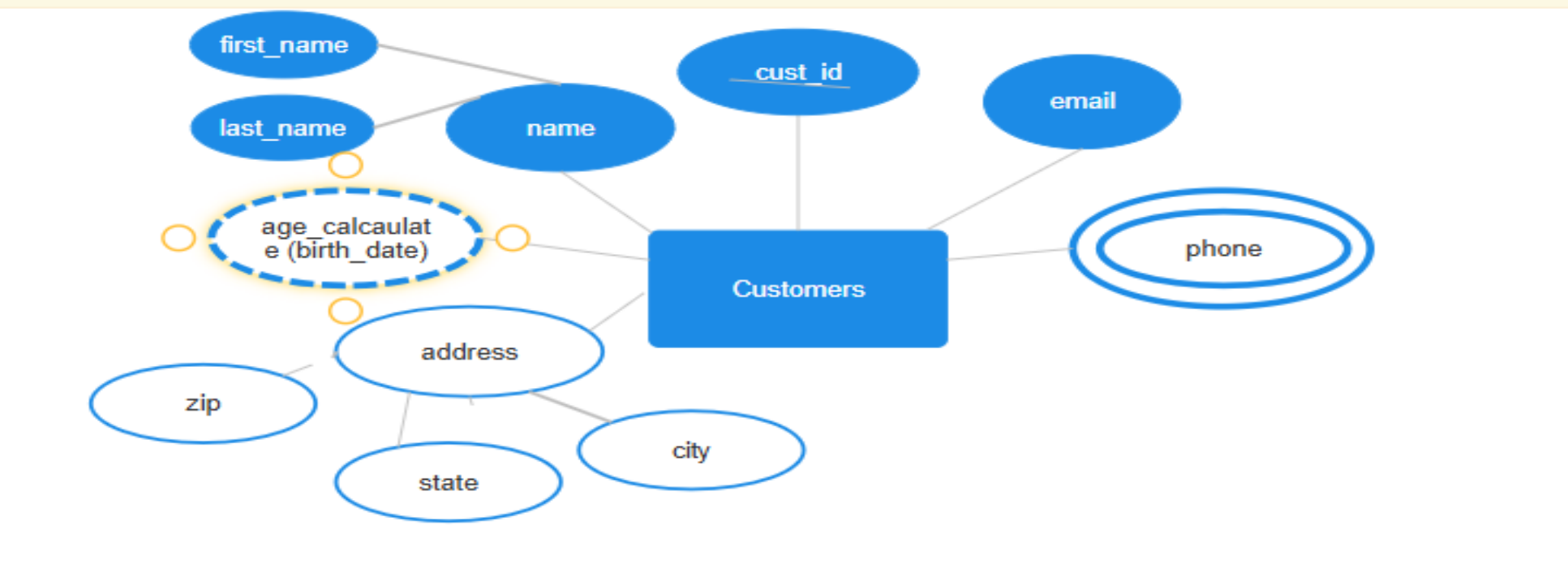
ERD with Shapes / Circles (Chen Notation):



Types of ERD (Entity-Relationship Diagrams)

ERD with Shapes / Circles (Chen Notation):

https://www.smartdraw.com/entity-relationship-diagram/er-diagram-tool.htm?srsltid=AfmBOopVHjUeF-11GV6UffX5PxmMdvMPs9wUmRzayTHOnsO0I_XuTp70



ERD with Shapes / Circles (Chen Notation):

1. Entity — CUSTOMER

The rectangle in the center. It represents a real-world object that we store data about. CUSTOMER is a strong entity — it exists independently and has its own unique identity. In Chen notation, a strong entity always uses a single-border rectangle.

2. Key Attribute — cust_id

A simple ellipse with an underlined name. It uniquely identifies each customer — no two customers can have the same cust_id. This is what becomes the Primary Key in the actual database table.

ERD with Shapes / Circles (Chen Notation):

3. Simple Attribute — name, email

A plain ellipse with no special styling. It holds a single, straightforward value per customer. name stores one name, email stores one email. These map directly to regular columns in a table.

4. Multivalued Attribute — phone

A double-border ellipse. It means one customer can have more than one value for this attribute — for example, a mobile number and a home number. In the actual database, this usually becomes a separate table (e.g. CUSTOMER_PHONES).

ERD with Shapes / Circles (Chen Notation):

5. Composite Attribute — address

An ellipse that branches into smaller ellipses (city and zip). It means the attribute is made up of smaller parts. Instead of storing address as one big text, we break it into meaningful pieces that can be searched or filtered individually.

6. Derived Attribute — age_account

A dashed ellipse. Its value is not stored in the database — it is calculated from another attribute. age_account (how long the account has existed) is calculated from the registration date. No need to store it separately because it changes over time automatically.

Key Concepts in ERD

1.Entities:

Represent objects or things in the system that we want to store information about.

Example: Customer, Product, Invoice.

2.Attributes (Fields)

Each entity has attributes, which are the specific pieces of data we want to store.

Example: Customer → Name, Email, Phone; Product → Name, Price, Category.

Key Concepts in ERD

3.Primary Key (PK):

A unique identifier for each record in an entity.

Example: Customer_ID, Product_ID.

4.Relationships:

Show how entities are connected.

One-to-One (1:1) → One record in Entity A relates to one record in Entity B.

One-to-Many (1:N) → One record in Entity A relates to multiple records in Entity B.

Many-to-Many (M:N) → Multiple records in Entity A relate to multiple records in Entity B (usually handled with a junction table).

Key Concepts in ERD

5.Foreign Key (FK) / Reference Fields

Attribute in one table that links to the primary key of another table.

Enables relational connections between entities.

6.Cardinality

Defines the number of instances of one entity that can or must be associated with instances of another entity.

Example: A Customer can have many invoices, but each Invoice belongs to one customer.

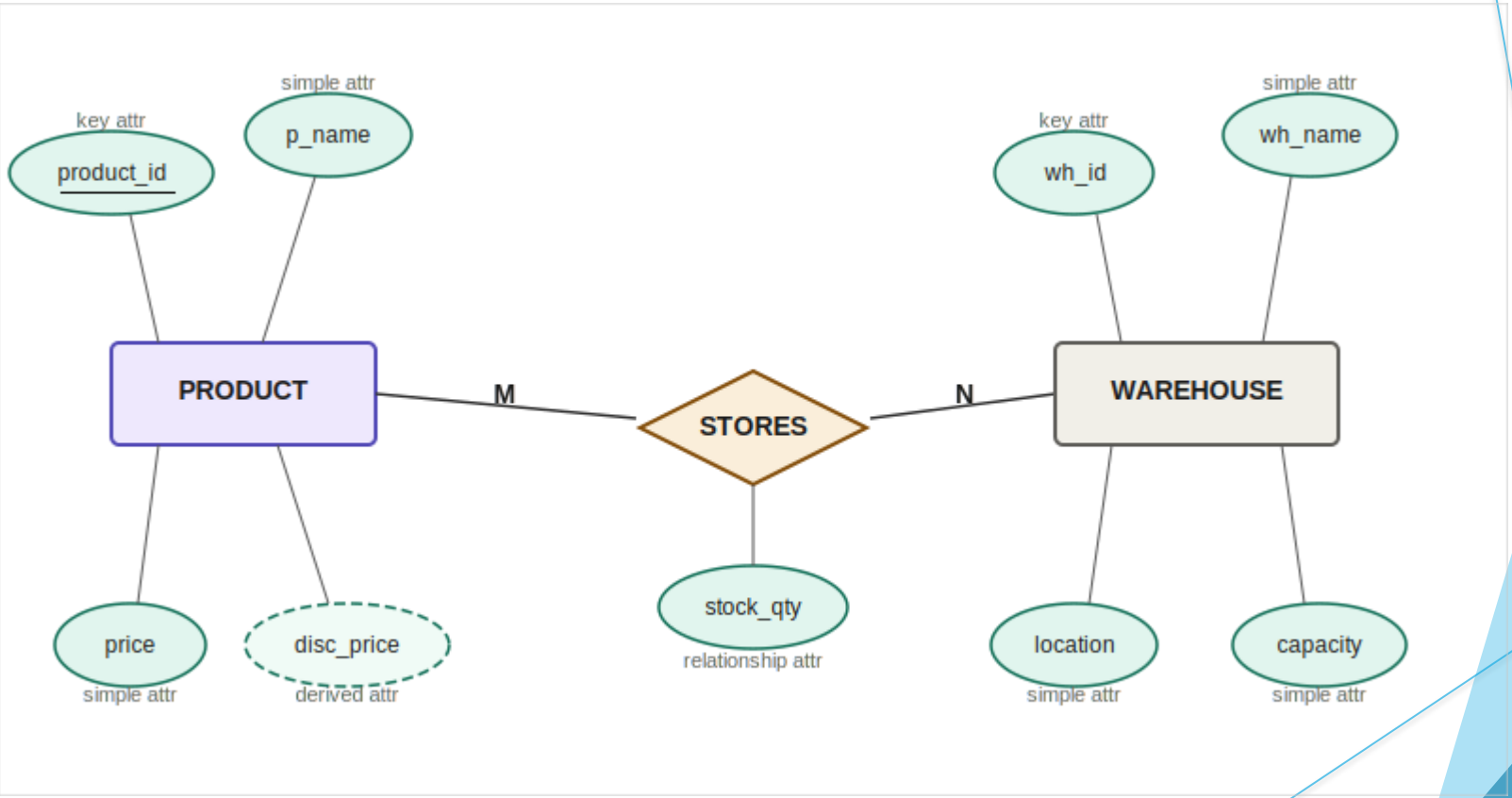
ERD with Shapes / Circles (Chen Notation):

Quick reference:

Shape	Type	Meaning
Single rectangle	Entity	Real-world object
Underlined ellipse	Key attribute	Unique identifier
Plain ellipse	Simple attribute	Single stored value
Double ellipse	Multivalued attribute	Can have multiple values
Ellipse with children	Composite attribute	Made of smaller parts
Dashed ellipse	Derived attribute	Calculated, not stored

Types of ERD (Entity-Relationship Diagrams)

ERD with Shapes / Circles (Chen Notation):



Types of ERD (Entity-Relationship Diagrams)

ERD with Shapes / Circles (Chen Notation):

ERD with Shapes / Circles (Chen Notation):

7.Relationship — STORES (Diamond)

The diamond shape connects two entities together. It represents the action or association between them. STORES means "a warehouse stores products." In Chen notation, every relationship between entities uses a diamond — never a line alone.

2. M:N Cardinality (Many-to-Many)

The M and N written on the lines. It means:

One PRODUCT can be stored in many warehouses

One WAREHOUSE holds many products

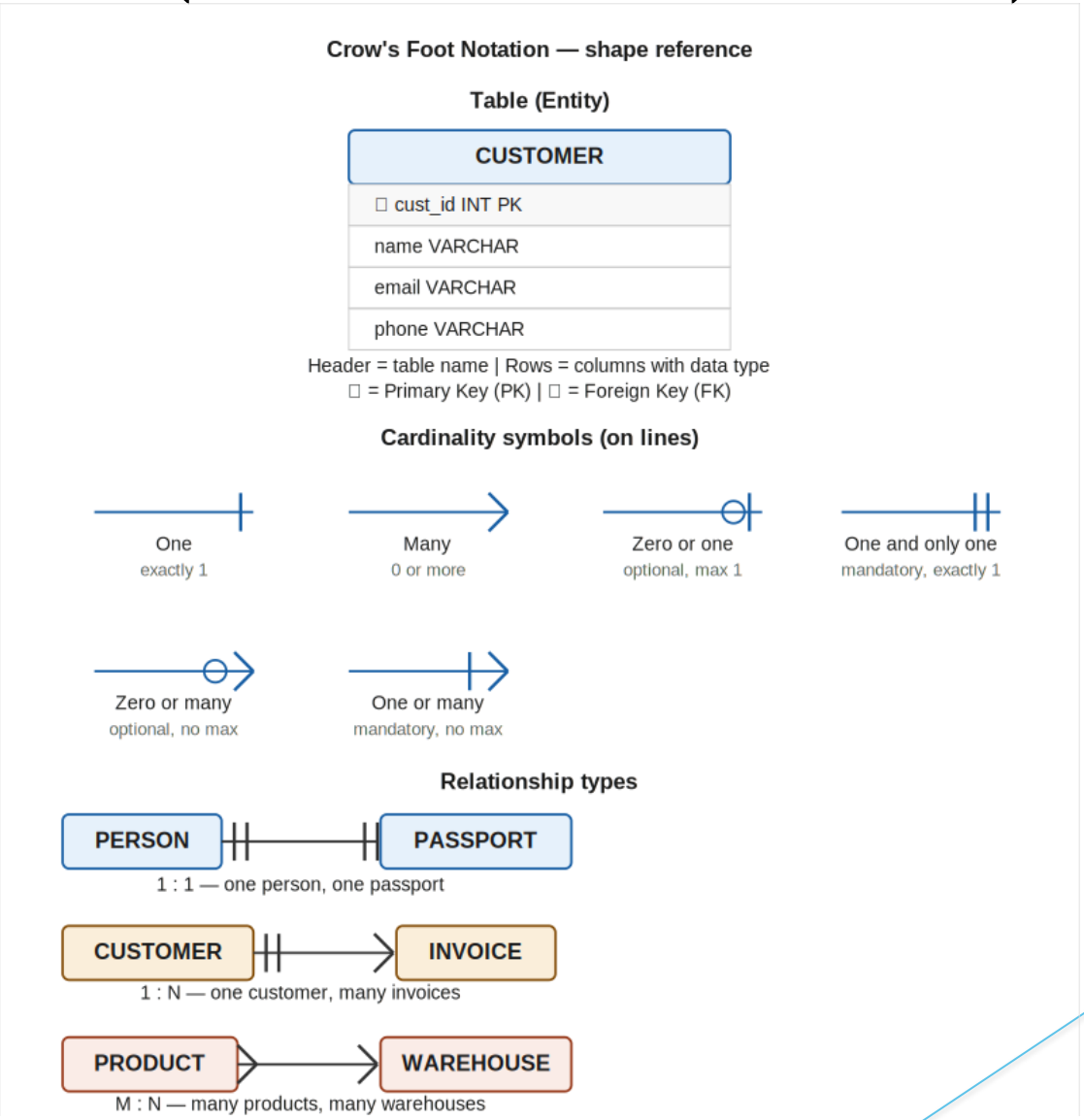
This is different from CUSTOMER → INVOICE which was 1:N (one customer, many invoices). M:N always requires a bridge table in the real database (e.g. WAREHOUSE_PRODUCT).

ERD with Shapes / Circles (Chen Notation):

3. Relationship Attribute — stock_qty

This is the biggest new concept. The ellipse hangs off the diamond, not off any entity. stock_qty doesn't belong to PRODUCT alone (a product has different quantities in different warehouses), and it doesn't belong to WAREHOUSE alone either. It only makes sense when both are combined — "how many units of this product are in this warehouse?" That value lives on the relationship itself.

2.ERD with Tables (Crow’s Foot Notation)



Importance of ERD in ServiceNow

1. Visualizes the Data Model

ServiceNow is built on tables and reference fields.

An ERD provides a clear map of entities and their relationships before implementation, making planning easier.

2. Reduces Implementation Errors

Understanding the correct relationships between tables (e.g., Customers → Invoices → Invoice Items) prevents mistakes when creating tables or reference fields.

Importance of ERD in ServiceNow

ServiceNow is built on tables and relationships (Reference Fields) → exactly like the concept of an ERD.

If you understand ERD, you can quickly see:

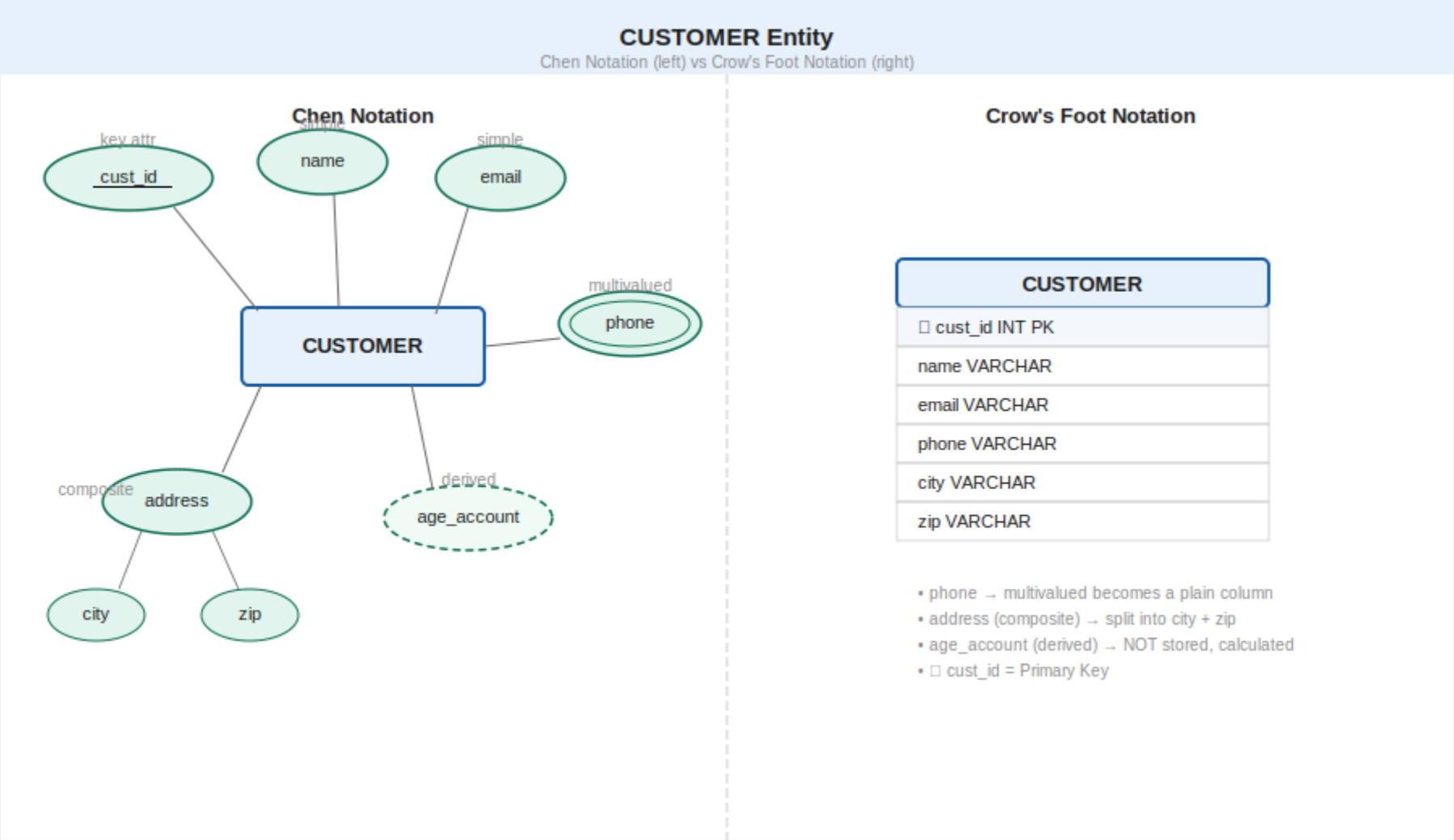
What fields each table has

How tables are related (1:N, M:N)

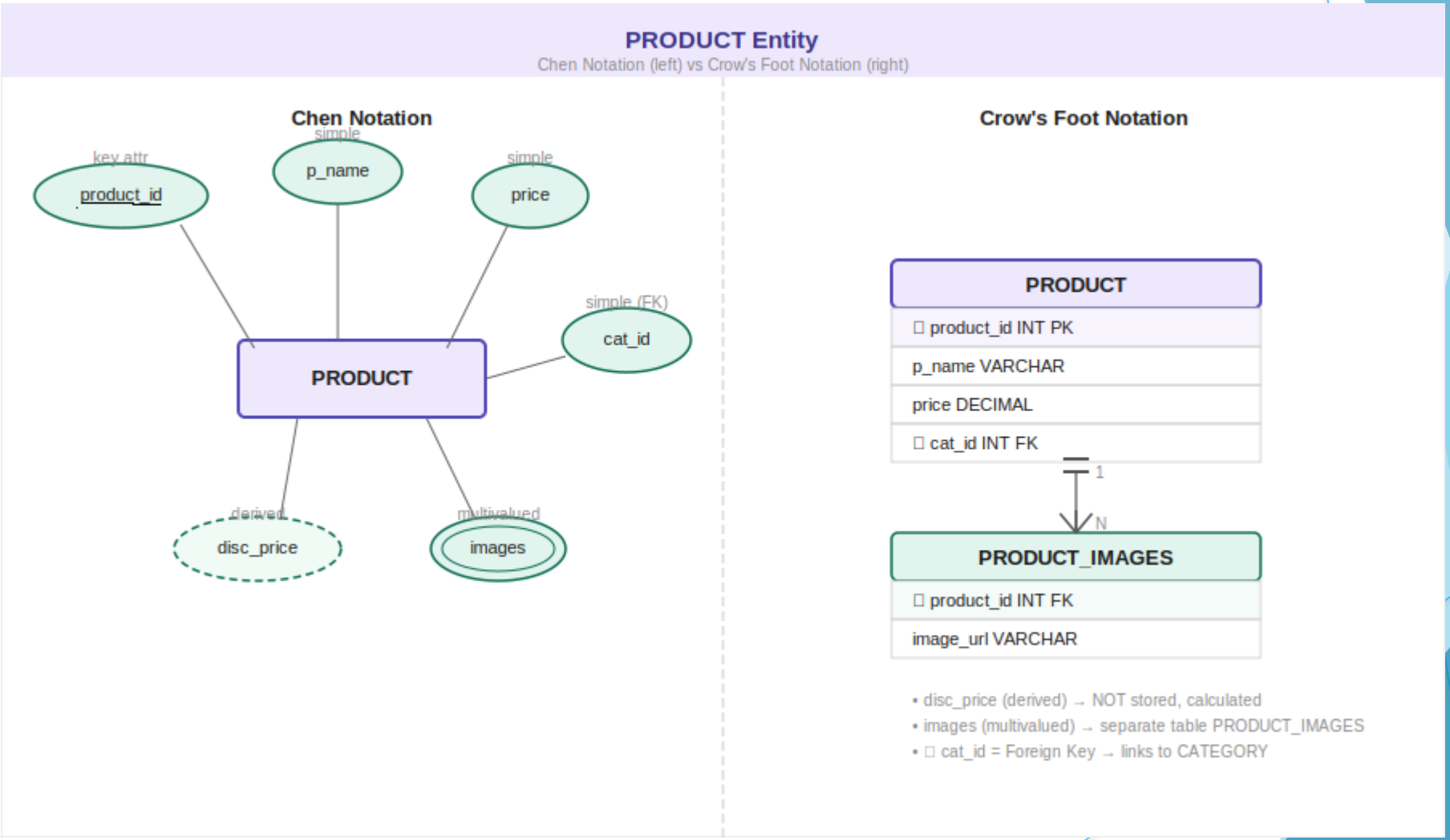
Where the foreign keys and reference fields are to pull data from

In other words, the ERD acts as a map that makes it much easier to work with the platform.

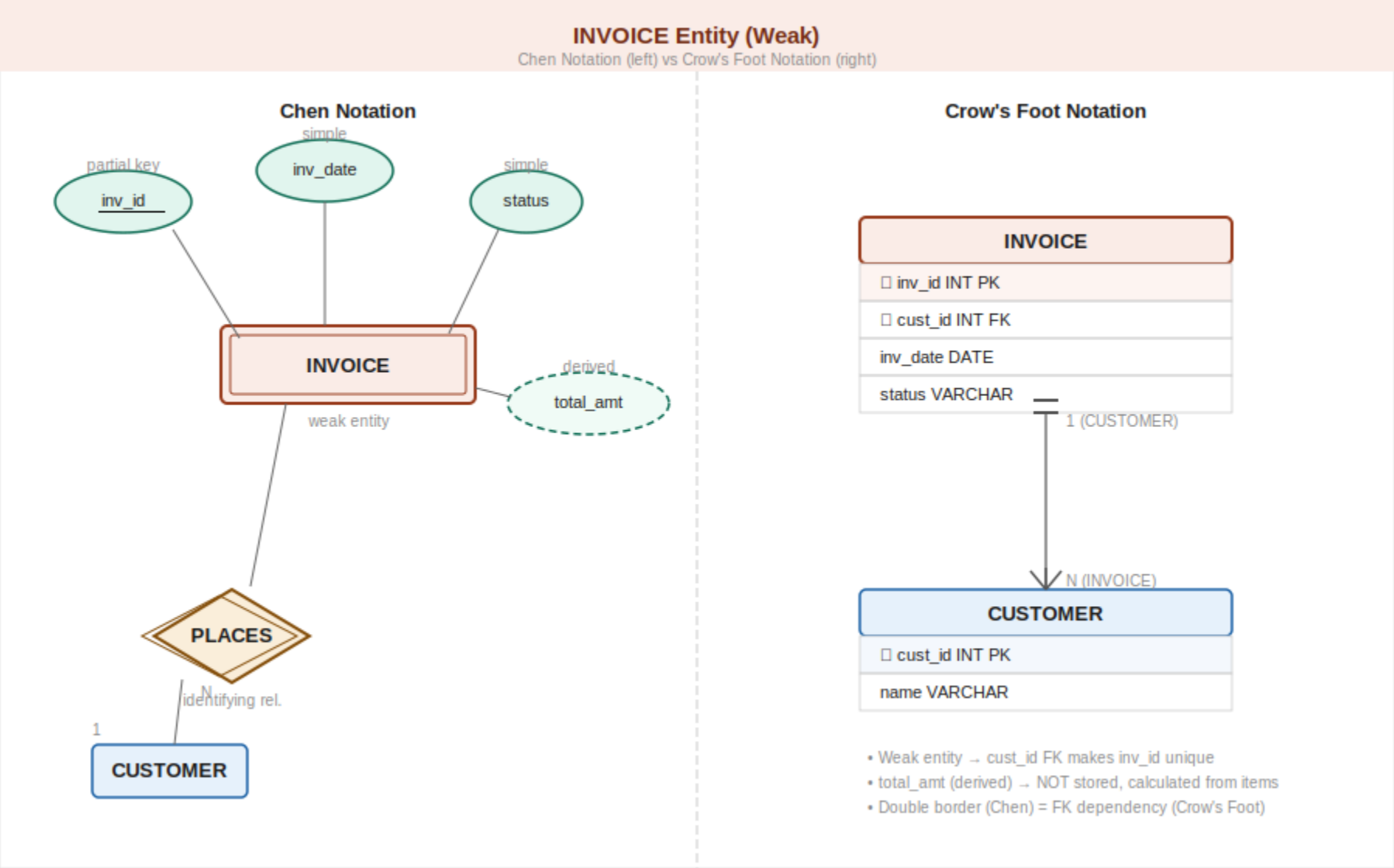
Key Concepts in ERD



Key Concepts in ERD



Key Concepts in ERD



Databases in ServiceNow

Databases in ServiceNow

ServiceNow is a platform that is built entirely on the concept of **databases**. At its core, everything in ServiceNow is stored and managed using **tables, records, and relationships**.

What is a Database in ServiceNow?
A database in ServiceNow is a collection of **tables** that store information about different parts of the system.
Each table represents a specific type of data, such as:
Users
Incidents
Requests
Configuration Items (CMDB)

Databases in ServiceNow

Tables, Rows, and Columns
Table → represents an entity (e.g., Users, Incidents)
Row (Record) → a single entry in the table
Column (Field) → a piece of data (e.g., name, email, status)
Example:
Table: sys_user
Columns: name, email, phone
Row: one specific user

Keys and Relationships
ServiceNow uses:
Primary Keys → unique identifier for each record
Reference Fields (Foreign Keys) → to connect tables
Example:
An incident is assigned to a user
This creates a relationship between tables

Databases in ServiceNow

Relationships in ServiceNow
Just like databases, ServiceNow supports:
One-to-Many (1:N)
One user → many incidents
Many-to-Many (M:N)
Users ↔ Groups
These relationships help organize and connect data efficiently.

Why Databases are Important in ServiceNow?
Databases allow ServiceNow to:
Store and organize data
Link related information
Automate processes
Generate reports and insights

Databases in ServiceNow

Special Case: CMDB
One of the most important databases in ServiceNow is the **CMDB (Configuration Management Database).**
It stores:
Servers
Applications
Devices
Their relationships
Helping understand the full IT system structure.

Databases in ServiceNow

Real Database Tables in ServiceNow
ServiceNow uses **real tables (like any database)**, and each table has a specific purpose.

1. Users Table

sys_user

This table stores **all users in the system.**

Important Fields:

sys_id → Primary Key (auto-generated)
name → User full name
email → User email
phone → Phone number
department → Reference to department

Used in:
Assigning incidents
Login system
Notifications

2. Incidents Table

incident

Stores **issues/problems reported by users**

Important Fields:

number → Incident number (e.g., INC001)
short_description
priority
state (New, In Progress, Resolved)
assigned_to → FK to sys_user

Relationship:
Many incidents → One user (1:N)

Databases in ServiceNow

3. Task Table (Parent Table)

task

This is a **base table** Other tables inherit from it:

incident

problem

change_request

Shared Fields:

number

state

assigned_to

created_on

Relation here:

Task = Parent → Incident = Child

4.Departments Table

cmn_department

Used to organize users

Databases in ServiceNow

How to Work with Database in ServiceNow (No SQL)
Important:
In ServiceNow, you **DON’T write SQL**
You work with **tables, forms, and reference fields** through the UI.

Concept core

SQL (Traditional DB)	ServiceNow
Write queries using SQL language	Use UI (forms, lists, filters)
Developer-focused	User + Admin friendly
Manual query writing	Visual configuration

Databases in ServiceNow

Tables and records

SQL	ServiceNow
Table	Table
Row	Record
Column	Field
Schema defined via SQL	Schema defined via UI

Query data

SQL	ServiceNow
`SELECT * FROM users`	Open list view
`WHERE`	Filter conditions
`AND / OR`	Filter builder
`ORDER BY`	Column sorting
`LIMIT`	"Show Top N"

Keys and relationship

SQL	ServiceNow
Primary Key (PK)	`sys_id` (auto-generated)
Foreign Key (FK)	Reference Field
JOIN between tables	Automatic via Reference
Constraints (FK, PK)	Enforced via platform

Joins

SQL JOIN	ServiceNow
Manual JOIN required	Automatic via reference
Complex syntax	Simple selection

Databases in ServiceNow

Aggregations function

`GROUP BY`	Reports
`COUNT()`	Count in reports
`SUM()`	Aggregation in reports
`HAVING`	Report filters

Data Insertion

SQL	ServiceNow
`INSERT INTO table`	Form → New Record
Manual queries	UI form

Databases in ServiceNow

If you need to **work with data using code**, not UI ??
You will use glideRecords

GlideRecord = the way to write queries in ServiceNow using scripting
It is the **replacement of SQL inside ServiceNow scripts**

Example for select:

```
var gr = new GlideRecord('incident');
gr.addQuery('state', '1'); // Open
gr.query();

while (gr.next()) {
    gs.info(gr.number);
}
```

INSERT:

```
var gr = new GlideRecord('incident');
gr.initialize();
gr.short_description = "Test Incident";
gr.insert();
```

Databases in ServiceNow

update:

```
var gr = new GlideRecord('incident');
gr.addQuery('number', 'INC001');
gr.query();

if (gr.next()) {
    gr.state = '2'; // In Progress
    gr.update();
}
```

delete:

```
var gr = new GlideRecord('incident');
gr.addQuery('number', 'INC001');
gr.query();

if (gr.next()) {
    gr.deleteRecord();
}
```

Auto-assign Incident to User

Inside Business Rule:

```
var user = new GlideRecord('sys_user');
user.addQuery('department', 'IT');
user.query();

if (user.next()) {
    current.assigned_to = user.sys_id;
}
```

Thank you